Corrigé type

Exercice 3 : Gestion d'un personnel universitaire

Programmation Orientée Objet en C++

Héritage · Polymorphisme · Classes abstraites · Méthodes virtuelles · Liaison dynamique

Énoncé résumé

Une université possède trois catégories de personnel : **enseignants**, **agents administratifs** et **techniciens**. Tous possèdent un matricule, un nom, un prénom et un salaire de base. Les enseignants perçoivent des indemnités spécifiques ; les agents administratifs et les techniciens bénéficient d'autres primes.

Travail demandé :

- Identifier la classe mère et les classes dérivées.
- Réaliser le diagramme de classes UML et représenter l'héritage.
- Implémenter les classes en C++ avec des méthodes virtuelles pour le calcul du salaire.
- Afficher les informations de chaque employé.
- Créer une collection contenant plusieurs catégories d'employés.

1. Identification des classes

Les trois catégories partagent les mêmes attributs (matricule, nom, prénom, salaire de base) : on les factorise dans une classe mère. Chaque catégorie y ajoute ses propres primes : ce sont les classes dérivées. La relation est bien de type **« est un »** (un enseignant *est un* employé), ce qui justifie l'héritage.

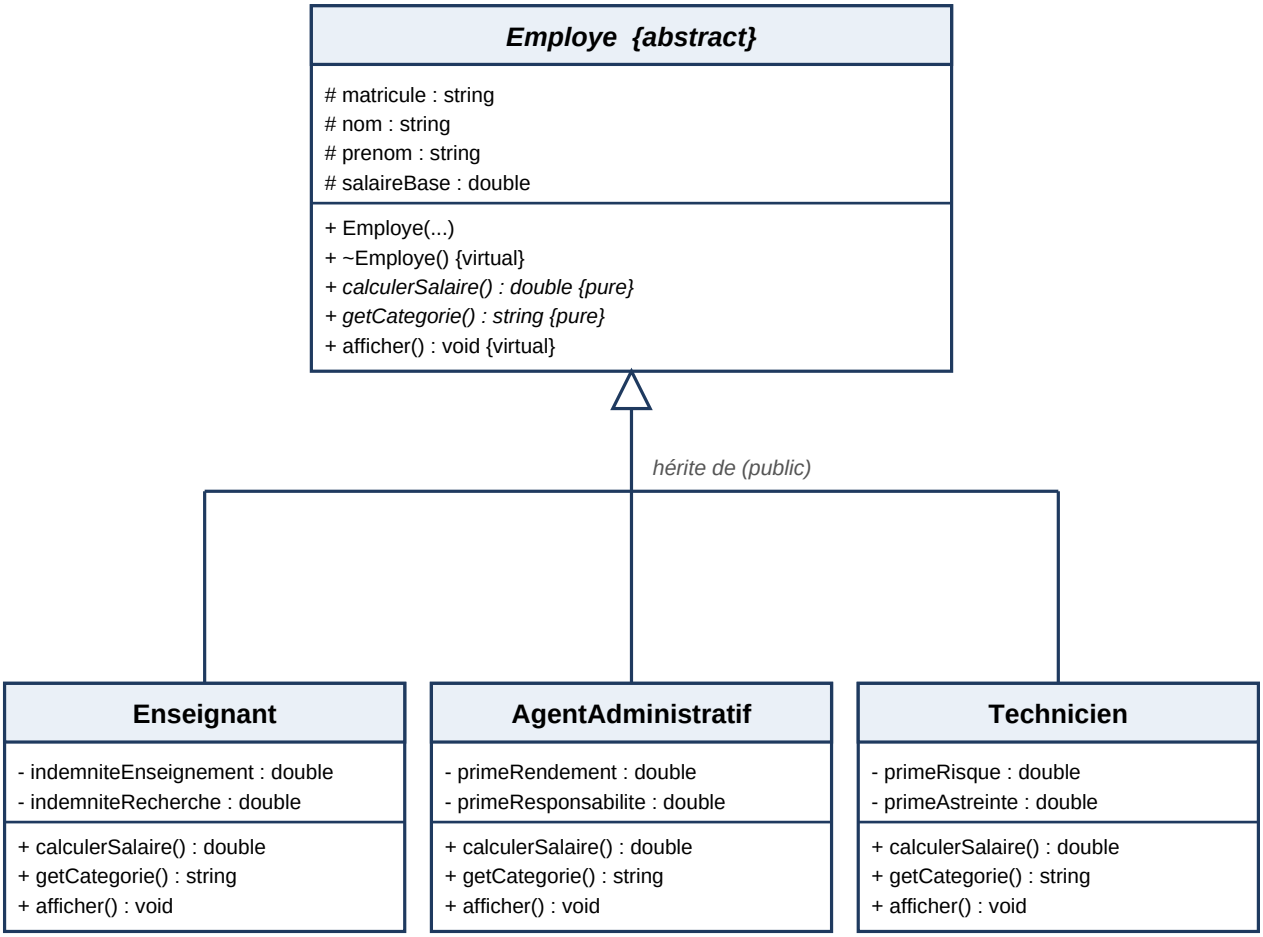
Classe	Rôle	Attributs propres	Calcul du salaire
Employe	Classe mère **abstraite**	matricule, nom, prenom, salaireBase	Non défini (virtuelle pure)
Enseignant	Classe dérivée	indemniteEnseignement, indemniteRecherche	base + indemnités enseignement + recherche
AgentAdministratif	Classe dérivée	primeRendement, primeResponsabilite	base + prime rendement + prime responsabilité
Technicien	Classe dérivée	primeRisque, primeAstreinte	base + prime risque + prime astreinte

Pourquoi une classe abstraite ? Il n'existe pas d'« employé » générique dans l'université : chacun appartient à une catégorie précise. De plus, la méthode `calculerSalaire()` ne peut pas être définie au niveau de la classe mère, car la règle dépend de la catégorie. On la déclare donc **virtuelle**

pure (`= 0`), ce qui rend `Employe` abstraite : elle ne peut plus être instanciée.

2. Diagramme de classes UML

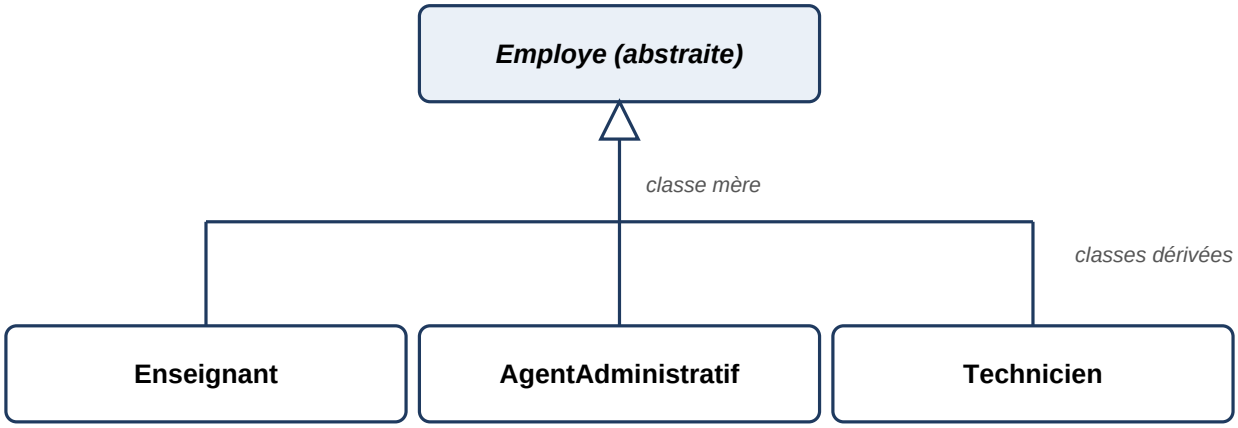
L'héritage est représenté par une flèche à **triangle creux** pointant vers la classe mère. Les noms et méthodes abstraits sont en *italique*. Notations : # protected, - private, + public.



Remarque : les attributs de `Employe` sont *protected* afin que les classes dérivées puissent les utiliser directement dans `calculerSalaire()`. Les primes spécifiques restent *private* (encapsulation).

3. Représentation des relations d'héritage

Avant d'écrire le code, on formalise la hiérarchie : une seule classe mère et trois classes dérivées **directes** (héritage simple, un seul niveau). Les trois classes dérivées sont indépendantes entre elles : un enseignant n'est ni un technicien ni un agent administratif.



Classe dérivée	Classe mère	Type d'héritage	Relation sémantique	Syntaxe C++
Enseignant	Employe	public, simple	« est un » employé	`class Enseignant : public Employe`
AgentAdministratif	Employe	public, simple	« est un » employé	`class AgentAdministratif : public Employe`
Technicien	Employe	public, simple	« est un » employé	`class Technicien : public Employe`

Pourquoi un héritage public ? L'héritage public conserve l'interface de la classe mère : un `Enseignant` peut être utilisé partout où un `Employe` est attendu (principe de substitution de Liskov). C'est la condition nécessaire au polymorphisme utilisé dans le `main`.

Ce qui est hérité : les attributs `matricule`, `nom`, `prenom`, `salaireBase` (accessibles car *protected*) et les méthodes communes. **Ce qui est redéfini :** `calculerSalaire()`, `getCategorie()` et `afficher()`.

4. Implémentation C++

Le projet est organisé en cinq fichiers : quatre fichiers d'en-tête (une classe chacun) et un fichier `main.cpp`.

4.1 Classe mère abstraite : Employe.h

```
#ifndef EMPLOYE_H
#define EMPLOYE_H

#include <iostream>
#include <string>

// Classe abstraite : racine de la hiérarchie
class Employe {
protected:
    std::string matricule;
    std::string nom;
    std::string prenom;
    double salaireBase;

public:
    Employe(const std::string& matricule, const std::string& nom,
            const std::string& prenom, double salaireBase)
        : matricule(matricule), nom(nom), prenom(prenom),
          salaireBase(salaireBase) {}

    // Destructeur virtuel : indispensable pour delete via Employe*
    virtual ~Employe() {}

    // Méthode virtuelle pure : chaque catégorie a sa règle de calcul
    virtual double calculerSalaire() const = 0;

    // Méthode virtuelle pure : nom de la catégorie
    virtual std::string getCategorie() const = 0;

    // Méthode virtuelle (redéfinissable) : affichage commun
    virtual void afficher() const {
        std::cout << "Categorie     : " << getCategorie() << "\n"
                  << "Matricule     : " << matricule << "\n"
                  << "Nom / Prenom  : " << nom << " " << prenom << "\n"
                  << "Salaire base  : " << salaireBase << " DA\n";
    }

    // Accesseurs
    std::string getMatricule() const { return matricule; }
    std::string getNom() const { return nom; }
    std::string getPrenom() const { return prenom; }
    double getSalaireBase() const { return salaireBase; }
};

#endif
```

4.2 Classe dérivée : Enseignant.h

```
#ifndef ENSEIGNANT_H
#define ENSEIGNANT_H

#include "Employe.h"

class Enseignant : public Employe {
private:
    double indemniteEnseignement;
    double indemniteRecherche;

public:
    Enseignant(const std::string& matricule, const std::string& nom,
               const std::string& prenom, double salaireBase,
               double indemniteEnseignement, double indemniteRecherche)
        : Employe(matricule, nom, prenom, salaireBase),
          indemniteEnseignement(indemniteEnseignement),
```

```
          indemniteRecherche(indemniteRecherche) {}

    double calculerSalaire() const override {
        return salaireBase + indemniteEnseignement + indemniteRecherche;
    }

    std::string getCategorie() const override { return "Enseignant"; }

    void afficher() const override {
        Employe::afficher();   // partie commune
        std::cout << "Indemn. enseignement : " << indemniteEnseignement << " DA\n"
                  << "Indemn. recherche    : " << indemniteRecherche << " DA\n"
                  << "SALAIRE TOTAL        : " << calculerSalaire() << " DA\n";
    }
};

#endif
```

4.3 Classe dérivée : AgentAdministratif.h

```
#ifndef AGENTADMINISTRATIF_H
#define AGENTADMINISTRATIF_H

#include "Employe.h"

class AgentAdministratif : public Employe {
private:
    double primeRendement;
    double primeResponsabilite;

public:
    AgentAdministratif(const std::string& matricule, const std::string& nom,
                       const std::string& prenom, double salaireBase,
                       double primeRendement, double primeResponsabilite)
        : Employe(matricule, nom, prenom, salaireBase),
          primeRendement(primeRendement),
          primeResponsabilite(primeResponsabilite) {}

    double calculerSalaire() const override {
        return salaireBase + primeRendement + primeResponsabilite;
    }

    std::string getCategorie() const override { return "Agent administratif"; }

    void afficher() const override {
        Employe::afficher();
        std::cout << "Prime rendement      : " << primeRendement << " DA\n"
                  << "Prime responsabilite : " << primeResponsabilite << " DA\n"
                  << "SALAIRE TOTAL        : " << calculerSalaire() << " DA\n";
    }
};

#endif
```

4.4 Classe dérivée : Technicien.h

```
#ifndef TECHNICIEN_H
#define TECHNICIEN_H

#include "Employe.h"

class Technicien : public Employe {
private:
    double primeRisque;
    double primeAstreinte;

public:
    Technicien(const std::string& matricule, const std::string& nom,
               const std::string& prenom, double salaireBase,
               double primeRisque, double primeAstreinte)
        : Employe(matricule, nom, prenom, salaireBase),
          primeRisque(primeRisque), primeAstreinte(primeAstreinte) {}
```

```
    double calculerSalaire() const override {
        return salaireBase + primeRisque + primeAstreinte;
    }

    std::string getCategorie() const override { return "Technicien"; }

    void afficher() const override {
        Employe::afficher();
        std::cout << "Prime de risque      : " << primeRisque << " DA\n"
                  << "Prime d'astreinte    : " << primeAstreinte << " DA\n"
                  << "SALAIRE TOTAL        : " << calculerSalaire() << " DA\n";
    }
};

#endif
```

4.5 Programme principal : main.cpp

```
#include <iostream>
#include <vector>
#include <memory>
#include "Enseignant.h"
#include "AgentAdministratif.h"
#include "Technicien.h"

int main() {
    // Collection hétérogène : pointeurs sur la classe mère
    std::vector<std::unique_ptr<Employe>> personnel;

    personnel.push_back(std::make_unique<Enseignant>(
        "E001", "Benali", "Karim", 90000, 25000, 15000));
    personnel.push_back(std::make_unique<Enseignant>(
        "E002", "Haddad", "Samia", 85000, 22000, 12000));
    personnel.push_back(std::make_unique<AgentAdministratif>(
        "A001", "Mansouri", "Yacine", 60000, 8000, 10000));
    personnel.push_back(std::make_unique<Technicien>(
        "T001", "Kaci", "Nadia", 55000, 7000, 5000));

    double masseSalariale = 0;

    std::cout << "===== PERSONNEL DE L'UNIVERSITE =====\n\n";
    for (const auto& e : personnel) {
        e->afficher();                    // liaison dynamique
        masseSalariale += e->calculerSalaire();   // liaison dynamique
        std::cout << "-------------------------------------\n";
    }

    std::cout << "\nMasse salariale totale : " << masseSalariale << " DA\n";
    return 0;
}   // les unique_ptr libèrent la mémoire (destructeur virtuel appelé)
```

4.6 Compilation et exécution

```
g++ -std=c++14 -Wall -o prog main.cpp
./prog
```

5. Résultat de l'exécution

Résultat obtenu avec le programme ci-dessus (compilé sans aucun avertissement) :

```
===== PERSONNEL DE L'UNIVERSITE =====

Categorie     : Enseignant
Matricule     : E001
Nom / Prenom  : Benali Karim
Salaire base  : 90000 DA
Indemn. enseignement : 25000 DA
Indemn. recherche    : 15000 DA
SALAIRE TOTAL        : 130000 DA
-------------------------------------
Categorie     : Enseignant
```

```
Matricule     : E002
Nom / Prenom  : Haddad Samia
Salaire base  : 85000 DA
Indemn. enseignement : 22000 DA
Indemn. recherche    : 12000 DA
SALAIRE TOTAL        : 119000 DA
-------------------------------------
Categorie     : Agent administratif
Matricule     : A001
Nom / Prenom  : Mansouri Yacine
Salaire base  : 60000 DA
Prime rendement      : 8000 DA
Prime responsabilite : 10000 DA
SALAIRE TOTAL        : 78000 DA
-------------------------------------
Categorie     : Technicien
Matricule     : T001
Nom / Prenom  : Kaci Nadia
Salaire base  : 55000 DA
Prime de risque      : 7000 DA
Prime d'astreinte    : 5000 DA
SALAIRE TOTAL        : 67000 DA
-------------------------------------

Masse salariale totale : 394000 DA
```

6. Explication des notions mobilisées

Héritage

`class Enseignant : public Employe` : la classe dérivée récupère les attributs et méthodes de la classe mère. Le constructeur de la dérivée appelle celui de la mère dans sa liste d'initialisation (`Employe(matricule, nom, ...)`).

Classe abstraite

Une classe possédant au moins une méthode virtuelle pure est abstraite. `Employe e;` provoque une erreur de compilation. Les classes dérivées deviennent concrètes seulement si elles redéfinissent **toutes** les méthodes pures.

Méthode virtuelle

Le mot-clé `virtual` indique que la méthode peut être redéfinie dans les classes dérivées et que l'appel sera résolu à l'exécution.

Redéfinition (override)

Chaque classe dérivée fournit sa propre version de `calculerSalaire()`, `getCategorie()` et `afficher()`. Le mot-clé `override` (C++11) fait vérifier la signature par le compilateur.

Polymorphisme et liaison dynamique

Dans la boucle du `main`, e est de type `Employe*`, mais l'appel `e->calculerSalaire()` exécute la version de l'objet réellement pointé (Enseignant, AgentAdministratif ou Technicien). Le choix est fait à l'exécution grâce à la table des méthodes virtuelles (vtable).

Destructeur virtuel

Sans `virtual ~Employe()`, détruire un objet dérivé via un pointeur de type `Employe*` serait un comportement indéfini (le destructeur de la dérivée ne serait pas appelé). C'est une règle à retenir : toute classe destinée à être héritée polymorphiquement doit avoir un destructeur virtuel.

Collection hétérogène

Un `vector<unique_ptr<Employe>>` peut contenir des objets de plusieurs classes dérivées. On stocke obligatoirement des **pointeurs** (ou références) : stocker des `Employe` par valeur provoquerait un « slicing » (perte de la partie dérivée), et c'est de toute façon impossible pour une classe abstraite.

7. Points d'attention (erreurs fréquentes)

- Oublier `virtual` sur la méthode de la classe mère : il n'y a alors plus de liaison dynamique.
- Oublier le destructeur virtuel de la classe mère.
- Stocker les objets par valeur dans le `vector` (slicing) au lieu de pointeurs.
- Ne pas redéfinir toutes les méthodes pures : la classe dérivée reste abstraite et ne peut pas être instanciée.
- Dans `afficher()` des classes dérivées, recopier le code commun au lieu d'appeler `Employe::afficher()` (duplication de code).